成员 1 集成决策（2026-10-20）

项	内容
决策人	成员 1（统稿 / controller）
日期	2026-10-20
适用范围	`main`、`agent/`、成员分支的合并顺序
状态	已决定，待各组按行动项执行

决策 1：`agent/` 定位 = **只读归档**

决定：只读，不再用于开发。

- 不在 `agent/` 下直接改代码或文档；所有改动走 `member-*` 分支。
- `agent/` 只用于查历史、对比、交接；不跑测试、不作为运行入口。
- `agent/README.md` 记录每个子目录的源分支 + 快照 commit + 日期；新增/替换归档必须单独评审。
- 根目录的集成代码与 `agent/` 归档是两份东西：集成看 root，历史看 `agent/`。

理由：

- `agent/` 的每个子树来自不同分支，历史独立；直接改会造成“archive 和 branch 两份真相”。
- 测试如果从子目录跑，会掩盖 root 集成的导入/路径/依赖问题。
- 只读能保证归档可追溯，不会被日常开发改脏。

决策 2：root 集成顺序 = ① 成员 5 → ② 成员 4 → ③ 成员 1 → ④ 成员 2

决定：按这个顺序合并，每步保持 `main` 可运行/可测。

步骤	合并内容	为什么在这个位置
① 成员 5	`app/domain/candidates.py`、`app/services/provider.py`、`binaryrepair_ai/`、`db/`、`schemas/`、基础测试	先立包结构、schema、provider 与依赖；后续都依赖它
② 成员 4	`app/domain/acceptance.py`、`outcomes.py`、验证设计、unit/integration 测试	只依赖 domain；先把单一判定点和测试基线放进来
③ 成员 1	**只合并 controller slice**：`app/services/jobs.py` + `app/db/`，不带 GUI	依赖成员 5 的 db/schema；适配后再接 API
④ 成员 2	`app/main.py`、`demo.py`、`i18n.py`、`templates/`、`static/`、GUI 测试	最后接真实 API/GUI；`app/main.py` 保持成员 2 原样，唯一入口

执行规则：

- 每步合并后跑对应测试（unit / integration / GUI），不把冲突留到下一步。
- 第 ③ 步不要直接把 `member1/jobs-minimal` 整分支合进来：那个分支为了跑测试恢复了 GUI，集成时只取 `app/services/jobs.py`、`app/db/`，或另开 `member1/controller-only`。
- 第 ④ 步保留成员 2 的 `app/main.py` 不改；controller/acceptance 只通过接口被调用，不复制判定逻辑。
- 任何一步出现"两套成功判定"或"skip 写成 pass"，立即回退该步。

为什么不是 2 先： GUI 不依赖真实后端也能跑 demo，但它会把 `app/main.py`/模板/测试一次性带进来，后续 domain/controller 的冲突都压到 GUI 上；先 5/4 立地基，再接 1，最后 2 接 UI，冲突最小。

决策 3：归档**冻结为历史快照**，不做定期 `git subtree pull`

决定：冻结；只有里程碑才重新归档。

- `agent/member-4-verification` 保持当前快照（源分支 `3a8aa4c`），不自动 `subtree pull`。
- 只有到里程碑（root 集成完成、提案提交、版本发布）才重新归档：
  - 用新的 `git subtree add` 替换该前缀，或新建快照目录；
  - 在 `agent/README.md` 记录源分支 commit / 日期 / 归档人。
- 需要最新代码时看源分支，不看 archive。
- 不在 archive 上跑测试；测试在 root 集成树或源分支跑。

理由：

- `subtree pull` 会不断产生合并提交与冲突，archive 和 root 双份维护，成本高。
- 归档的价值是"可追溯的历史快照"，不是"永远最新"。
- 冻结后 `agent/` 不会成为第二个开发树，符合决策 1。

`agent/member-4-verification` 已过时是预期现象；在 `agent/README.md` 标"snapshot @ 3a8aa4c · 2026-10-18"即可，不需要追平。

行动项

#	谁	做什么	完成标准
1	成员 1	拆 `member1/controller-only`（只含 `app/services/jobs.py` + `app/db/`）	可单独合入步骤 ③
2	成员 1	在 `agent/README.md` 加快照表（源分支 / commit / 日期 / 只读声明）	归档状态一眼可见
3	成员 4	在 `member1/jobs-minimal` 重跑 `pytest tests/integration/test_identity_state.py -rs`	真实 pass/skip 数，skip 不算 pass
4	成员 2	保持 `app/main.py` 原样，等步骤 ④	controller 不改 GUI 入口

#	谁	做什么	完成标准
5	成员 3/5	按提案骨架第 5 节填证据	每条带路径/命令/四级标签

一句话结论

`agent/` **只读；集成按 5 → 4 → 1（仅 controller）→ 2；归档冻结，不做定期 pull，里程碑再重归档。**